

Conception et Optimisation d'une Base de Données de Gestion de la Recherche

TP1 - Gestion de Bases de Données et Data Warehouse

Master 1 Data Engineering (M1 DENG)

Université de Corse

Réalisé par :

Nom Prénom 1

Nom Prénom 2

Nom Prénom 3

Date de rendu : 12 novembre 2025

Table des matières

1	Introduction	4
1.1	Contexte du projet	4
1.2	Objectifs	4
1.3	Organisation du rapport	4
2	Partie 1 - Conception de la Base de Données	5
2.1	Méthodologie et utilisation de l'IA générative	5
2.1.1	Outils utilisés	5
2.1.2	Démarche adoptée	5
2.1.3	Résultats	5
2.1.4	Analyse critique	5
2.2	Interprétation du cahier des charges	6
2.2.1	Entités principales identifiées	6
2.2.2	Relations entre entités	6
2.2.3	Hypothèses et choix de conception	6
2.2.4	Dictionnaire des attributs	7
2.3	Schéma relationnel	10
2.3.1	Vue d'ensemble	10
2.3.2	Contraintes CHECK	11
3	Partie 2 - Peuplement de la Base de Données	12
3.1	Méthodologie	12
3.1.1	Outils utilisés	12
3.1.2	Démarche générale	12
3.2	Architecture du script Python	12
3.2.1	Structure du code	12
3.2.2	Gestion de l'intégrité référentielle	13
3.3	Génération des données par table	13
3.3.1	Institutions	13
3.3.2	Laboratoires	13
3.3.3	Chercheurs	14
3.3.4	Projets	14
3.3.5	Contrats	14
3.3.6	Publications	14
3.3.7	Jeux de données	15
3.4	Cohérence des données générées	15
3.4.1	Statistiques des données générées	15
3.5	Exemples de code	16
3.5.1	Génération des projets	16
3.5.2	Génération des chercheurs	16

4	Partie 3 - Gestion des Utilisateurs et Création de Vues	18
4.1	Stratégie de sécurité	18
4.1.1	Objectifs	18
4.1.2	Architecture des rôles	18
4.2	Définition des rôles	18
4.2.1	Rôle Chercheur	18
4.2.2	Rôle Data Manager	18
4.2.3	Rôle Administrateur	18
4.3	Vues créées	18
4.3.1	Vue 1 : Mes_Projets	18
4.3.2	Vue 2 : Metadata_Jeux_Donnees	19
4.3.3	Vue 3 : Publications_Par_Laboratoire	19
4.3.4	Vue 4 : Contrats_Actifs	19
4.3.5	Vue 5 : Projets_Budget_Global	19
4.4	Attribution des privilèges	19
4.4.1	Privilèges par rôle	19
4.4.2	Exemples de scripts SQL	20
5	Partie 4 - Requêtes et Optimisation	21
5.1	Méthodologie d'optimisation	21
5.1.1	Démarche	21
5.2	Requête R1 - Jeux de données par projet	21
5.2.1	Énoncé	21
5.2.2	Version 1	21
5.2.3	Version 2	21
5.2.4	Analyse des performances	21
5.2.5	Plans d'exécution	21
5.2.6	Optimisation choisie	21
5.3	Requête R2 - Projets sans chercheurs peu productifs	22
5.3.1	Énoncé	22
5.3.2	Version 1	22
5.3.3	Version 2	22
5.3.4	Analyse des performances	22
5.3.5	Plans d'exécution	22
5.3.6	Optimisation choisie	22
5.4	Requête R3 - Laboratoires conformes	22
5.4.1	Énoncé	22
5.4.2	Version 1	22
5.4.3	Version 2	23
5.4.4	Analyse des performances	23
5.4.5	Plans d'exécution	23
5.4.6	Optimisation choisie	23
5.5	Synthèse des optimisations	23
5.5.1	Index créés	23
5.5.2	Gains de performance	23

6	Partie 5 - Triggers et Procédures Stockées	24
6.1	Triggers	24
6.1.1	Trigger 1 : Limite de participants à un projet	24
6.1.2	Trigger 2 : Vérification DMP d'un jeu de données	24
6.2	Procédures stockées et fonctions	24
6.2.1	Fonction 1 : Nombre de publications d'un projet	24
6.2.2	Procédure 2 : Préparation du bilan d'un projet	25
6.2.3	Fonction 3 : Fiche projet	25
6.2.4	Procédure 4 : Archivage des contrats échus	25
6.3	Documentation du code	26
6.3.1	Conventions de nommage	26
6.3.2	Commentaires	26
7	Conclusion	27
7.1	Synthèse du travail réalisé	27
7.2	Apports pédagogiques	27
7.2.1	Compétences techniques acquises	27
7.2.2	Utilisation de l'IA générative	27
7.3	Difficultés rencontrées	27
7.4	Perspectives d'amélioration	27
A	Scripts SQL	28
A.1	Script de création du schéma	28
A.2	Script d'insertion des données	28
A.3	Script de gestion des utilisateurs	28
A.4	Scripts des requêtes optimisées	28
A.5	Scripts des triggers et procédures	28
B	Code Python	28
B.1	Script de génération de données	28
C	Captures d'écran	28
C.1	Plans d'exécution	28
C.2	Tests des procédures	28
D	Dictionnaire de données	28

1. Introduction

1.1. Contexte du projet

Ce projet s'inscrit dans le cadre du TP1 du module de Gestion de Bases de Données et Data Warehouse. L'objectif est de concevoir et implémenter une base de données relationnelle pour la gestion des données de recherche d'une université.

1.2. Objectifs

Les objectifs principaux, comme énoncés dans le sujet, sont :

- Concevoir un schéma relationnel cohérent pour gérer les institutions, laboratoires, projets, chercheurs, contrats, publications et jeux de données
- Peupler la base avec des données fictives réalistes
- Implémenter une gestion sécurisée des accès via des rôles et des vues
- Optimiser les requêtes complexes
- Mettre en œuvre des triggers et procédures stockées

Ces objectifs seront développés dans les différentes parties de ce rapport.

1.3. Organisation du rapport

Ce rapport présente successivement :

- La conception de la base de données (Partie 1) qui s'appuie sur l'utilisation d'outils d'IA générative.
- Le peuplement de la base (Partie 2)
- La gestion des utilisateurs et des vues (Partie 3)
- L'optimisation des requêtes (Partie 4)
- Les triggers et procédures stockées (Partie 5)

2. Partie 1 - Conception de la Base de Données

2.1. Méthodologie et utilisation de l'IA générative

2.1.1 Outils utilisés

La phase de conception a été assistée par l'agent de code Claude Sonnet 4.5 par le biais de l'extension Visual Studio Code "GitHub Copilot Chat".

Cet outil d'IA générative a principalement permis, à partir d'instructions et du contexte de l'environnement de développement, de proposer des suggestions de code, des structures de données, et d'aider à la modélisation conceptuelle, le tout par le biais de création de fichiers de façon automatique.

2.1.2 Démarche adoptée

1. Analyse du cahier des charges
2. Identification des entités et relations
3. Validation et raffinement avec l'IA
4. Analyse critique des propositions et corrections

2.1.3 Résultats

L'IA a permis de générer une première version du schéma relationnel, incluant les tables, attributs, types de données et contraintes d'intégrité.

Après lui avoir donné le contexte de l'exercice et les exigences (par un simple copier-coller du cahier des charges), nous avons obtenu :

- Un schéma relationnel complet
- Des descriptions détaillées des tables
- Des suggestions de contraintes d'intégrité
- Les requêtes SQL de création
- Un fichier .dbml pour visualiser le MCD
- Des règles métier et triggers de validation (que nous avons supprimés par la suite car non demandés, et non conformes au sujet du TP)

2.1.4 Analyse critique

L'utilisation de l'IA générative nous a été extrêmement utile pour obtenir une base solide, cohérente dans un temps très réduit. Les suggestions étaient globalement pertinentes et conformes au cahier des charges.

Nous avons rencontré cependant une limitation qui ne nous a pas réellement pénalisés : l'IA a pris volontairement la décision de simplifier la relation entre deux entités (Chercheur et Projet) en optant pour une relation 1-N au lieu de N-N (un chercheur ne pouvait être que dans un seul projet.. Or ce n'est pas réaliste et assez contraignant), ce qui n'était pas conforme au sujet. Nous avons corrigé cela en y ajoutant une table d'association.

2.2. Interprétation du cahier des charges

2.2.1 Entités principales identifiées

- Institution
- Laboratoire
- Projet
- Chercheur
- Contrat
- Publication
- Jeu de données

2.2.2 Relations entre entités

- **Institution** → **Laboratoire** : Une institution héberge un ou plusieurs laboratoires (1-N). Chaque laboratoire appartient à une seule institution ;
- **Laboratoire** → **Projet** : Un laboratoire pilote un ou plusieurs projets (1-N). Chaque projet a un laboratoire pilote unique ;
- **Laboratoire** → **Chercheur** : Un laboratoire regroupe plusieurs chercheurs (1-N). Chaque chercheur est rattaché à un laboratoire ;
- **Projet** ↔ **Chercheur** : Relation N-N modélisée par la table d'association **projet_chercheur**. Un projet peut impliquer plusieurs chercheurs, et un chercheur peut participer à plusieurs projets. Cette table permet de gérer le rôle, la période de participation et le pourcentage d'effort de chaque chercheur sur chaque projet ;
- **Projet** → **Chercheur (responsable)** : Un projet a un chercheur responsable (1-1 optionnel). Un chercheur peut être responsable de plusieurs projets ;
- **Projet** → **Contrat** : Un projet peut être financé par plusieurs contrats (1-N). Chaque contrat finance un seul projet ;
- **Contrat** → **Jeu _Donnees** : Un contrat peut générer plusieurs jeux de données (1-N). Chaque jeu de données est associé à un unique contrat ;
- **Chercheur** → **Jeu _Donnees** : Un chercheur peut produire plusieurs jeux de données (1-N). Chaque jeu de données a un auteur principal unique ;
- **Publication** ↔ **Chercheur** : Relation N-N modélisée par la table d'association **publication_auteur**. Une publication peut avoir plusieurs auteurs, et un chercheur peut publier plusieurs articles. L'attribut **ordre_auteur** permet de gérer l'ordre de signature.

2.2.3 Hypothèses et choix de conception

Selon nos connaissances sur le domaine de la recherche académique, nous avons fait les choix suivants :

- Un chercheur peut participer à plusieurs projets simultanément ;
- Un projet peut avoir plusieurs chercheurs responsables (co-responsables) mais pour simplifier, nous avons choisi un seul responsable principal ;
- Les publications ne sont pas directement rattachées aux projets, mais aux chercheurs. Cela reflète mieux la réalité où les chercheurs publient indépendamment de leurs projets ;

- Les jeux de données sont liés aux contrats car ils résultent souvent des financements obtenus ;
- Les types énumérés (ENUM) ont été définis pour les attributs avec des valeurs limitées (type d'institution, type de contrat, statut DMP).

2.2.4 Dictionnaire des attributs

Le dictionnaire nous a permis de clarifier le rôle de chaque table et attribut, ainsi que les contraintes associées.

Types énumérés `type_institution` :

- `universite` : Université
- `organisme_recherche` : Organisme de recherche
- `partenaire_privé` : Partenaire privé

`type_contrat` :

- `ANR` : Agence Nationale de la Recherche
- `H2020` : Horizon 2020 (programme européen)
- `Region` : Financement régional
- `Europe` : Financement européen
- `Autre` : Autre type de financement

`statut_dmp` :

- `brouillon` : DMP en cours de rédaction
- `soumis` : DMP soumis pour validation
- `valide` : DMP validé

Table Institution Institution de rattachement (université, organisme de recherche ou partenaire privé).

Attribut	Type	Description
id	bigint	Identifiant unique (PK, auto-increment)
nom	text	Nom de l'institution (NOT NULL)
type_institution	type_institution	Type d'institution (NOT NULL, enum)
adresse	text	Adresse postale de l'institution

TABLE 1 – Attributs de la table Institution. Mise en forme LaTeX par une IA générative.

Table Laboratoire Laboratoire de recherche rattaché à une institution.

Attribut	Type	Description
id	bigint	Identifiant unique (PK, auto-increment)
nom	text	Nom du laboratoire (NOT NULL)
id_institution	bigint	FK vers Institution (NOT NULL)

TABLE 2 – Attributs de la table Laboratoire. Mise en forme LaTeX par une IA générative.

Table Projet Projet de recherche structurant dirigé par un chercheur unique.

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
titre	text	Titre du projet (NOT NULL)
description	text	Description détaillée
discipline	text	Discipline scientifique (NOT NULL)
budget_annuel_eur	decimal(12,2)	Budget annuel ≥ 0 (NOT NULL)
date_debut	date	Date de début (NOT NULL)
date_fin	date	Date de fin $\geq$ date_debut
id_laboratoire_pilote	bigint	FK vers Laboratoire (NOT NULL)
id_chercheur_responsable	bigint	FK vers Chercheur

TABLE 3 – Attributs de la table Projet. Mise en forme LaTeX par une IA générative.

Table Chercheur Chercheur affilié à un laboratoire.

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
prenom	text	Prénom (NOT NULL)
nom	text	Nom (NOT NULL)
email	text	Email professionnel (NOT NULL, UNIQUE)
orcid	varchar(19)	Identifiant ORCID (UNIQUE)
discipline	text	Discipline de recherche
id_laboratoire	bigint	FK vers Laboratoire (NOT NULL)

TABLE 4 – Attributs de la table Chercheur. Mise en forme LaTeX par une IA générative.

Table Projet_Chercheur Table associative N-N entre projets et chercheurs.

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
id_projet	bigint	FK vers Projet (NOT NULL)
id_chercheur	bigint	FK vers Chercheur (NOT NULL)
role	text	Rôle dans le projet
date_debut	date	Date de début de participation
date_fin	date	Date de fin de participation
charge_pct	decimal(5,2)	Pourcentage d'effort (0-100)
is_principal	boolean	Chercheur principal (défaut : false)

TABLE 5 – Attributs de la table Projet_Chercheur. Mise en forme LaTeX par une IA générative.

Table Contrat Contrat de financement d'un projet.

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
type_contrat	type_contrat	Type de contrat (NOT NULL, enum)
financeur	text	Organisme financeur (NOT NULL)
intitule	text	Intitulé du contrat (NOT NULL)
montant_eur	decimal(14,2)	Montant total ≥ 0 (NOT NULL)
duree_mois	int	Durée en mois (> 0)
date_debut	date	Date de début (NOT NULL)
date_fin	date	Date de fin (NOT NULL)
id_projet	bigint	FK vers Projet (NOT NULL)
statut_dmp	statut_dmp	Statut DMP (NOT NULL)
date_validation_dmp	date	Date validation DMP
url_document_dmp	text	URL du document DMP

TABLE 6 – Attributs de la table Contrat. Mise en forme LaTeX par une IA générative.

Table Publication Publication scientifique (métadonnées uniquement).

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
titre	text	Titre de la publication (NOT NULL)
doi	text	Digital Object Identifier (UNIQUE)
date_publication	date	Date de publication
nb_pages	int	Nombre de pages (> 0)
url_externe	text	URL vers le document (ex : HAL)

TABLE 7 – Attributs de la table Publication. Mise en forme LaTeX par une IA générative.

Table Publication_Auteur Table associative N-N entre publications et chercheurs.

Attribut	Type	Description
id_publication	bigint	FK vers Publication (NOT NULL)
id_chercheur	bigint	FK vers Chercheur (NOT NULL)
ordre_auteur	int	Ordre d'auteur (> 0, NOT NULL)

TABLE 8 – Attributs de la table Publication_Auteur. Mise en forme LaTeX par une IA générative.

Table Jeu_Donnees Jeu de données produit dans le cadre d'un contrat.

Attribut	Type	Description
id	bigint	Identifiant unique (PK)
id_contrat	bigint	FK vers Contrat (NOT NULL)
description	text	Description du dataset (NOT NULL)
id_auteur	bigint	FK vers Chercheur (NOT NULL)
conditions_accès	text	Conditions d'accès
licence	text	Licence d'utilisation
date_creation	date	Date de création du jeu de données
date_depot	date	Date de dépôt officiel
url_externe	text	URL vers l'entrepôt

TABLE 9 – Attributs de la table Jeu_Donnees. Mise en forme LaTeX par une IA générative.

2.3. Schéma relationnel

2.3.1 Vue d'ensemble

Présentation du schéma relationnel complet (diagramme). Voir Figure 1.

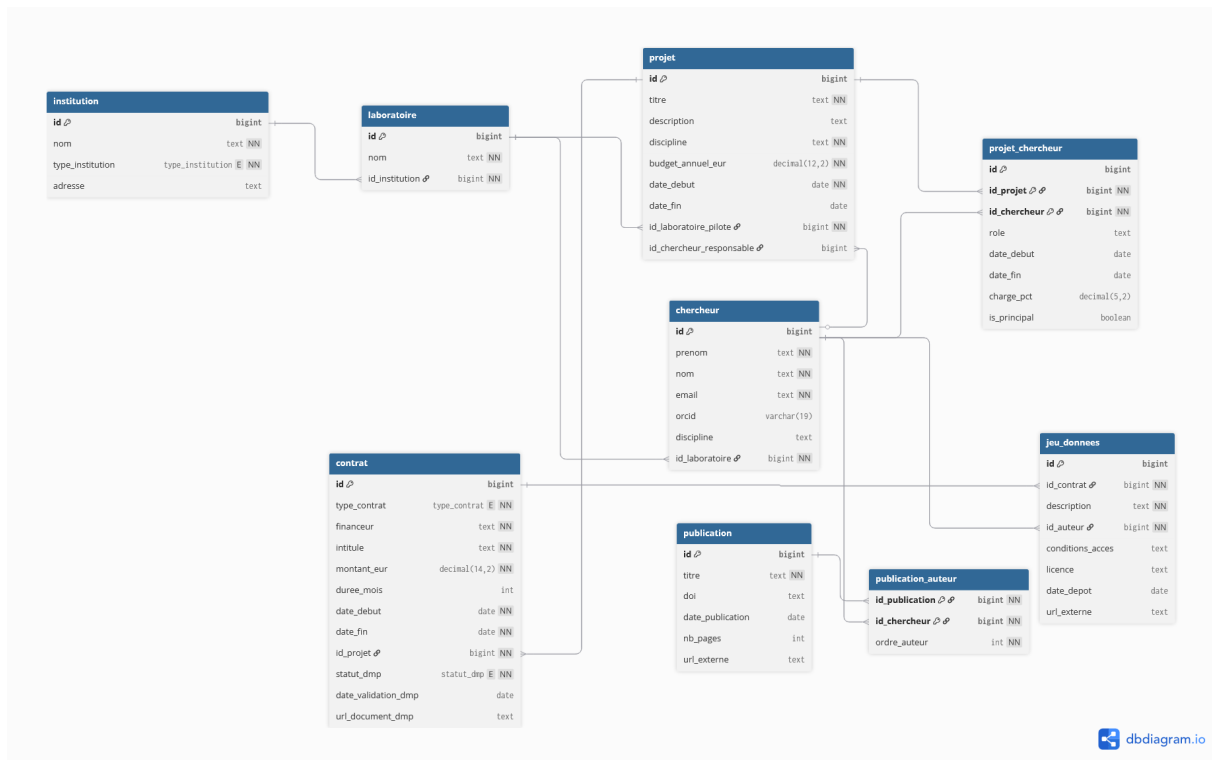


FIGURE 1 – Schéma relationnel de la base de données. Effectué sur [DBDiagram.io](https://dbdiagram.io). En utilisant le langage DBML.

2.3.2 Contraintes CHECK

- Vérification que $\text{budget_annuel_eur} \geq 0$
- Vérification que $\text{montant_eur} \geq 0$
- Vérification que $\text{date_fin} \geq \text{date_debut}$
- Validation du statut DMP et de ses champs associés

3. Partie 2 - Peuplement de la Base de Données

3.1. Méthodologie

3.1.1 Outils utilisés

- Python 3.x
- Bibliothèque Faker
- PostgreSQL

3.1.2 Démarche générale

Pour mener à bien ce générateur de données, nous avons tout d'abord analysés les types de données nécessaires pour chaque table, ainsi que les contraintes d'intégrité référentielle.

Dans notre cas, nous avons besoin de générer des emails, des dates cohérentes, des montants financiers, des identifiants uniques, de faux noms et prénoms, des titres de projets et publications, etc.

Pour ce faire, la simple utilisation de la Bibliothèque Faker nous a permis de générer la majorité des données de manière réaliste. Le reste des données spécifiques (comme les types de contrats, le statut DMP, etc.) ont été gérées par des listes prédéfinies.

Ensuite, nous avons structuré le script Python pour générer les données dans un ordre respectant les dépendances entre les tables (par exemple, générer les institutions avant les laboratoires, etc.).

L'analyse des contraintes d'intégrité est la partie la plus délicate, car il faut s'assurer que les clés étrangères pointent vers des enregistrements existants, et que les données respectent les contraintes de dates et autres règles métier. Le tout dans un code maintenable, modulaire et clair, pour faciliter les modifications futures.

3.2. Architecture du script Python

3.2.1 Structure du code

Le script Python est organisé en plusieurs fonctions modulaires qui génèrent les données de manière séquentielle en respectant les dépendances :

- **Fonctions utilitaires** :
 - `_quote_sql()` : Échappement sécurisé des valeurs SQL ;
 - `gen_orcid()` : Génération d'identifiants ORCID au format standard ;
 - `random_date_between()` : Génération de dates cohérentes dans un intervalle.
- **Fonction d'en-tête** :
 - `write_header()` : Inclusion du schéma et initialisation de la transaction.
- **Fonctions de génération par table** (dans l'ordre des dépendances) :
 - `generate_institutions()` : Création des institutions avec types variés ;
 - `generate_laboratoires()` : Rattachement aux institutions avec noms uniques ;
 - `generate_chercheurs()` : Génération avec emails et ORCID uniques ;
 - `generate_projets()` : Attribution de responsables et dates cohérentes ;
 - `generate_contrats()` : Gestion des statuts DMP et retour des contrats validés ;

- `generate_jeux_donnees()` : Respect des contraintes DMP (dépôt uniquement si validé) ;
- `generate_publications()` : Création avec DOI uniques et auteurs multiples ;
- `generate_projet_chercheur()` : Table associative N-N avec rôles et charges.
- **Fonction principale** :
 - `main()` : Orchestration de l'ensemble du processus de génération.

Le script utilise un système de retour des identifiants générés pour garantir la cohérence référentielle entre les tables. Chaque fonction retourne la liste des IDs créés, qui sont ensuite utilisés comme clés étrangères par les fonctions suivantes.

3.2.2 Gestion de l'intégrité référentielle

Nous avons mis en place plusieurs mécanismes pour garantir l'intégrité référentielle et le respect des différentes contraintes :

- **Propagation des IDs** : Chaque fonction de génération retourne la liste des identifiants créés, qui sont passés en paramètre aux fonctions suivantes
- **Gestion des contraintes d'unicité** :
 - Utilisation de sets Python (`emails_used`, `orcid_ids_used`, `dois_used`) pour éviter les doublons ;
 - Système de tentatives multiples (jusqu'à 100 tentatives pour trouver une valeur unique sinon mise à `null`) pour générer des valeurs uniques ;
 - Mécanisme de fallback avec suffixes numériques en cas d'échec.
- **Respect des contraintes temporelles** :
 - Vérification que `date_fin` $\geq$ `date_debut` pour les projets et contrats ;
 - Génération de dates de validation DMP postérieures au début du contrat ;
 - Cohérence des dates de participation chercheur-projet.
- **Respect des règles métier** :
 - Dépôt de jeux de données uniquement pour les contrats avec DMP validé ;
 - Attribution de chercheurs responsables existants aux projets ;
 - Génération de noms de laboratoires uniques par institution.
- **Mise à jour des séquences** : Réinitialisation automatique des séquences PostgreSQL après chaque table avec `setval()` pour permettre des insertions manuelles ultérieures.

3.3. Génération des données par table

3.3.1 Institutions

- **Nombre généré** : Paramétrable, par défaut `max(3,  $\lfloor rows/10 \rfloor$ )` ;
- Génération via `fake.company()` avec types variés (université, organisme de recherche, partenaire privé) ;
- **Exemple** : "Martin Technologies", type "partenaire_prive", adresse "12 rue de la Paix, 75002 Paris".

3.3.2 Laboratoires

- **Nombre généré** : `max(5,  $\lfloor rows/8 \rfloor$ )` ;

- Rattachement d'une institution aléatoire avec garantie d'unicité des noms par institution ;
- **Exemple** : "Deliver innovative Lab", rattaché à l'institution 2.

3.3.3 Chercheurs

- **Nombre généré** : $\max(20, \text{rows})$ (paramètre principal) ;
- **Attributs générés** :
 - Email unique construit à partir du prénom et nom ;
 - ORCID unique généré pour 50% des chercheurs (format XXXX-XXXX-XXXX-XXXX) ;
 - Discipline parmi 5 domaines (Physique, Biologie, Informatique, Chimie, Mathématiques).
- **Cohérence** : Rattachement aléatoire mais cohérent à un laboratoire existant ;
- **Exemple** : Marie Dubois, marie.dubois@example.com, ORCID 1234-5678-9012-3456, discipline Informatique.

3.3.4 Projets

- **Nombre généré** : $\max(10, \lfloor \text{rows}/5 \rfloor)$;
- Sélection aléatoire d'un chercheur existant pour le chercheur responsable (peut être NULL) ;
- Date de début entre -5 ans et aujourd'hui, date de fin entre date_debut et +3 ans ;
- Montants réalistes du budget (entre 10 000€ et 500 000€) ;
- **Exemple** : "Optimize cross-platform networks", budget 125 000€, du 2020-03-15 au 2023-12-31.

3.3.5 Contrats

- **Nombre généré** : $\max(10, \lfloor \text{rows}/4 \rfloor)$;
- Distribution des types réaliste entre ANR, H2020, Région, Europe, Autre ;
- Montants Entre 10 000€ et 2 000 000€ ;
- **Gestion DMP** :
 - Statuts pondérés : 50% brouillon, 30% soumis, 20% validé ;
 - Date de validation et URL uniquement pour les contrats validés ;
 - Retour séparé des contrats validés pour les jeux de données.

3.3.6 Publications

- **Nombre généré** : $\max(10, \lfloor \text{rows}/5 \rfloor)$;
- Génération unique de DOI pour 40% des publications (format 10.XXXX/XXXXXX) ;
- Entre 1 et 5 auteurs par publication avec ordre de signature ;
- Métadonnées : Date de publication, nombre de pages, URL externe (HAL) ;
- **Exemple** : "Impact of climate on biodiversity", DOI 10.1234/abc123, 3 auteurs, 15 pages.

3.3.7 Jeux de données

- **Nombre généré** : $\max(5, \lfloor \text{rows}/6 \rfloor)$;
- Date de dépôt générée uniquement pour les contrats avec DMP validé (40% des cas) ;
- Distribution des Licences entre CC-BY, CC0, ODbL ou NULL ;
- Conditions d'accès entre Ouvert, restreint ou sur demande ;
- **Exemple** : Dataset associé au contrat validé 15, licence CC-BY, déposé le 2024-05-12.

3.4. Cohérence des données générées

Les données générées respectent plusieurs niveaux de cohérence :

1. **Cohérence temporelle** : Toutes les dates respectent les contraintes CHECK ($\text{date_fin} \geq \text{date_debut}$)
2. **Cohérence référentielle** : Toutes les clés étrangères pointent vers des enregistrements existants grâce au système de propagation des IDs
3. **Cohérence métier** :
 - Les jeux de données avec `date_depot` sont liés à des contrats DMP validés
 - Les chercheurs responsables de projets appartiennent à des laboratoires existants
 - Les auteurs de publications sont des chercheurs référencés
 - L'ordre des auteurs est unique par publication
4. **Unicité garantie** : Emails, ORCID, DOI sont uniques grâce aux mécanismes de détection de doublons
5. **Réalisme des données** : Utilisation de Faker avec locale française pour des noms, adresses et domaines cohérents

3.4.1 Statistiques des données générées

Tableau récapitulatif du nombre d'enregistrements par table pour un paramètre `nb_rows` fixé à 50 000.

Table	Nombre d'enregistrements
Institution	5 000
Laboratoire	6 250
Chercheur	50 000
Projet	10 000
Contrat	12 460
Publication	10 000
Publication_Auteur	29 818
Jeu_Donnees	6 150

TABLE 10 – Statistiques des données générées pour 50 000 chercheurs.

3.5. Exemples de code

3.5.1 Génération des projets

```

1 def generate_projets(f, fake: Faker, laboratoires: List[int], chercheurs
  : List[int], count: int) -> List[int]:
2     """Genere les projets et retourne la liste des IDs."""
3     projets = []
4
5     for pid in range(1, count + 1):
6         titre = fake.catch_phrase()
7         description = fake.paragraph(nb_sentences=3)
8         discipline = random.choice(["Physique", "Biologie", "
          Informatique", "Chimie", "Mathematiques"])
9         budget = round(random.uniform(10000, 500000), 2)
10        d_debut = fake.date_between(start_date='-5y', end_date='today')
11        d_fin = None if random.random() < 0.2 else fake.date_between(
          start_date=d_debut, end_date='+3y')
12        id_labo = random.choice(laboratoires)
13        id_resp = random.choice(chercheurs) if chercheurs else None
14        projets.append(pid)
15        f.write(f"INSERT INTO univ_recherche.projet (id, titre,
          description, discipline, budget_annuel_eur, date_debut,
          date_fin, id_laboratoire_pilote, id_chercheur_responsable)
          VALUES ({pid}, {_quote_sql(titre)}, {_quote_sql(description)}
          }, {_quote_sql(discipline)}, {budget}, {_quote_sql(d_debut)},
          {_quote_sql(d_fin)}, {id_labo}, {_quote_sql(id_resp)});\n")
16
17    f.write("SELECT setval(pg_get_serial_sequence('univ_recherche.projet
          ','id'), COALESCE((SELECT MAX(id) FROM univ_recherche.projet),0))
          ;\n")
18    return projets

```

Listing 1 – Exemple de génération des projets

3.5.2 Génération des chercheurs

```

1 def generate_chercheurs(f, fake: Faker, laboratoires: List[int], count:
  int) -> List[int]:
2     """Genere les chercheurs et retourne la liste des IDs."""
3     chercheurs = []
4     emails_used = set()
5     orcid_ids_used = set()
6
7     for cid in range(1, count + 1):
8         prenom = fake.first_name()
9         nom = fake.last_name()
10
11        # Generer un email unique
12        attempts = 0
13        while attempts < 100:
14            if attempts == 0:
15                email = (prenom + "." + nom + "@" + fake.domain_name()).
                  lower()

```

```

16         else:
17             email = (prenom + "." + nom + str(attempts) + "@" + fake
18                     .domain_name()).lower()
19             if email not in emails_used:
20                 emails_used.add(email)
21                 break
22             attempts += 1
23         else:
24             # Fallback: email avec ID unique
25             email = f"chercheur{cid}@" + fake.domain_name()
26             emails_used.add(email)
27
28             # Generer un ORCID unique ou NULL
29             orcid = None
30             if random.random() < 0.5:
31                 attempts = 0
32                 while attempts < 100:
33                     orcid_candidate = gen_orcid(fake)
34                     if orcid_candidate not in orcids_used:
35                         orcids_used.add(orcid_candidate)
36                         orcid = orcid_candidate
37                         break
38                 attempts += 1
39
40             discipline = random.choice(["Physique", "Biologie", "
41                                         Informatique", "Chimie", "Mathematiques"])
42             id_labo = random.choice(laboratoires)
43             chercheurs.append(cid)
44             f.write(f"INSERT INTO univ_recherche.chercheur (id, prenom, nom,
45                     email, orcid, discipline, id_laboratoire) VALUES ({cid}, {
46                         _quote_sql(prenom)}, {_quote_sql(nom)}, {_quote_sql(email)},
47                         {_quote_sql(orcid)}, {_quote_sql(discipline)}, {id_labo});\n"
48                     )
49
50             f.write("SELECT setval(pg_get_serial_sequence('univ_recherche.
51                     chercheur','id'), COALESCE((SELECT MAX(id) FROM univ_recherche.
52                     chercheur),0));\n")
53
54             return chercheurs

```

Listing 2 – Exemple de génération des chercheurs

4. Partie 3 - Gestion des Utilisateurs et Création de Vues

4.1. Stratégie de sécurité

4.1.1 Objectifs

Pour pouvoir mettre en oeuvre une base de données réaliste, elle doit être sécurisée. Nous avons alors, en suivant rigoureusement les besoins de l'énoncé, mis en place une stratégie de sécurité basée sur la gestion des rôles et des privilèges.

4.1.2 Architecture des rôles

Nous avons défini trois rôles principaux pour gérer les accès à la base de données :

- **Chercheur** : Accès restreint aux données liées à ses propres projets et publications.
- **Data Manager** : Accès élargi aux métadonnées des jeux de données et DMP, mais sans droits de suppression.
- **Administrateur** : Accès complet à toutes les données et fonctionnalités de la base.

4.2. Définition des rôles

4.2.1 Rôle Chercheur

- Description : Accès restreint aux données du chercheur
- Privilèges : SELECT, INSERT, UPDATE sur ses propres données
- Restrictions : Pas d'accès aux données des autres chercheurs

4.2.2 Rôle Data Manager

- Description : Accès élargi aux métadonnées
- Privilèges : SELECT sur toutes les tables, INSERT/UPDATE sur jeux de données et DMP
- Restrictions : Pas de suppression de données

4.2.3 Rôle Administrateur

- Description : Accès complet
- Privilèges : ALL PRIVILEGES
- Responsabilités : Gestion complète de la base

4.3. Vues créées

4.3.1 Vue 1 : Mes_Projets

- Objectif : Permettre à un chercheur de voir ses projets
- Utilisateurs cibles : Chercheurs
- Contenu : Projets auxquels le chercheur participe

```

1 CREATE OR REPLACE VIEW v_mes_projets AS
2 SELECT
3     p.id AS projet_id,
4     p.titre,
5     p.discipline,
6     p.date_debut,
7     p.date_fin,
8     p.budget_annuel_eur,
9     l.nom AS laboratoire,
10    CONCAT(resp.prenom, ' ', resp.nom) AS responsable,
11    pc.role AS mon_role,
12    pc.charge_pct
13 FROM projet p
14 JOIN laboratoire l ON p.id_laboratoire_pilote = l.id
15 JOIN projet_chercheur pc ON p.id = pc.id_projet
16 LEFT JOIN chercheur resp ON p.id_chercheur_responsable = resp.id
17 WHERE pc.id_chercheur = (SELECT id FROM chercheur WHERE email =
    current_user);

```

Listing 3 – Vue Mes_Projets

4.3.2 Vue 2 : Metadata_Jeux_Donnees

- Objectif : Vue complète des métadonnées des jeux de données
- Utilisateurs cibles : Data Managers
- Contenu : Toutes les informations sur les jeux de données

4.3.3 Vue 3 : Publications_Par_Laboratoire

- Objectif : Statistiques des publications par laboratoire
- Utilisateurs cibles : Administrateurs, Data Managers

4.3.4 Vue 4 : Contrats_Actifs

- Objectif : Liste des contrats en cours
- Utilisateurs cibles : Tous les rôles

4.3.5 Vue 5 : Projets_Budget_Global

- Objectif : Vue agrégée des budgets par projet
 - Utilisateurs cibles : Administrateurs
- Voir Annexe pour le code SQL complet des vues.

4.4. Attribution des privilèges

4.4.1 Privilèges par rôle

Tableau récapitulatif des privilèges accordés à chaque rôle.

Vue/Table	Chercheur	Data Manager	Admin
mes_projets	SELECT	SELECT	ALL
metadata_jeux_donnees	-	SELECT, INSERT	ALL
publications_par_labo	-	SELECT	ALL
contrats_actifs	SELECT	SELECT	ALL
projets_budget_global	-	-	ALL

TABLE 11 – Attribution des privilèges par rôle

4.4.2 Exemples de scripts SQL

```

1  -- Role Chercheur
2  CREATE ROLE role_chercheur;
3
4  -- Role Data Manager
5  CREATE ROLE role_data_manager;
6
7  -- Role Administrateur
8  CREATE ROLE role_administrateur;
9
10 -- Creation des vues, etc. (voir sections precedentes)
11
12 -- Privileges pour ADMINISTRATEUR
13 GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA univ_recherche TO
    role_administrateur;
14 GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA univ_recherche TO
    role_administrateur;
15 GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA univ_recherche TO
    role_administrateur;
16
17 -- Privileges pour DATA_MANAGER
18 GRANT SELECT ON ALL TABLES IN SCHEMA univ_recherche TO role_data_manager
    ;
19 GRANT INSERT, UPDATE, DELETE ON contrat, jeu_donnees TO
    role_data_manager;
20 GRANT SELECT ON v_metadonnees_contrats, v_synthese_donnees_projet,
    v_tableau_bord_projets TO role_data_manager;
21 GRANT SELECT ON v_publications_chercheur TO role_data_manager;
22
23 -- Privileges pour CHERCHEUR
24 GRANT SELECT ON projet, chercheur, laboratoire, institution, publication
    TO role_chercheur;
25 GRANT SELECT ON v_mes_projets, v_mes_donnees, v_publications_chercheur
    TO role_chercheur;
26 GRANT INSERT ON publication, publication_auteur, jeu_donnees TO
    role_chercheur;
27 GRANT UPDATE ON jeu_donnees TO role_chercheur;

```

Listing 4 – Création des rôles et attribution des privilèges

5. Partie 4 - Requêtes et Optimisation

5.1. Méthodologie d'optimisation

5.1.1 Démarche

1. Écriture de plusieurs versions de chaque requête
2. Analyse des plans d'exécution
3. Comparaison des performances et des méthodes utilisées par le SGBD
4. Création d'index ciblés
5. Validation des améliorations

5.2. Requête R1 - Jeux de données par projet

5.2.1 Énoncé

Nombre total de jeux de données déposés par projet et par année civile, ainsi que la moyenne du délai de dépôt (en jours) pour les projets ayant impliqué > 5 chercheurs depuis 2018.

5.2.2 Version 1

```
1 -- Code SQL Version 1
```

Listing 5 – R1 - Version 1

5.2.3 Version 2

```
1 -- Code SQL Version 2
```

Listing 6 – R1 - Version 2

5.2.4 Analyse des performances

- Temps d'exécution Version 1 : XX ms
- Temps d'exécution Version 2 : XX ms
- Opérations coûteuses identifiées

5.2.5 Plans d'exécution

Captures d'écran et analyse des plans EXPLAIN ANALYZE.

5.2.6 Optimisation choisie

Justification du choix de la version retenue et description des index créés.

```
1 -- Code SQL des index
```

Listing 7 – Index pour R1

5.3. Requête R2 - Projets sans chercheurs peu productifs

5.3.1 Énoncé

Pour le laboratoire LISA en 2024, donner la liste des projets et le nom du responsable qui n'ont aucun chercheur dont le nombre de publications en 2024 est strictement inférieur à la moyenne.

5.3.2 Version 1

```
1 -- Code SQL Version 1
```

Listing 8 – R2 - Version 1

5.3.3 Version 2

```
1 -- Code SQL Version 2
```

Listing 9 – R2 - Version 2

5.3.4 Analyse des performances

Comparaison des deux versions.

5.3.5 Plans d'exécution

Analyse détaillée.

5.3.6 Optimisation choisie

Justification et index.

5.4. Requête R3 - Laboratoires conformes

5.4.1 Énoncé

Laboratoires n'ayant eu aucun jeu de données non conforme en 2024 (sans licence ou sans date_depot).

5.4.2 Version 1

```
1 -- Code SQL Version 1
```

Listing 10 – R3 - Version 1

5.4.3 Version 2

```
1 -- Code SQL Version 2
```

Listing 11 – R3 - Version 2

5.4.4 Analyse des performances

Comparaison détaillée.

5.4.5 Plans d'exécution

Captures et analyse.

5.4.6 Optimisation choisie

Justification et index.

5.5. Synthèse des optimisations

5.5.1 Index créés

Liste récapitulative de tous les index créés avec leur justification.

Table	Colonnes	Justification
jeu_donnees	(date_depot)	Filtrage par année
chercheur	(id_projet)	Jointures fréquentes
publication_auteur	(id_chercheur, date)	Comptage publications

TABLE 12 – Index créés pour l'optimisation

5.5.2 Gains de performance

Tableau comparatif des temps d'exécution avant et après optimisation.

6. Partie 5 - Triggers et Procédures Stockées

6.1. Triggers

6.1.1 Trigger 1 : Limite de participants à un projet

Objectif Vérifier lors de l'insertion d'une participation qu'on ne dépasse pas la capacité maximale du projet.

Implémentation

```
1 -- Code SQL du trigger
```

Listing 12 – Trigger limite participants

Tests Description des tests effectués et résultats.

6.1.2 Trigger 2 : Vérification DMP d'un jeu de données

Objectif Empêcher le passage d'un jeu de données en statut "déposé" si le DMP du contrat n'est pas validé.

Implémentation

```
1 -- Code SQL du trigger
```

Listing 13 – Trigger vérification DMP

Tests Scénarios de test et résultats.

6.2. Procédures stockées et fonctions

6.2.1 Fonction 1 : Nombre de publications d'un projet

Objectif Calculer le nombre de publications pour un projet donné et une année donnée.

Signature

```
1 CREATE FUNCTION nb_publications_projet(  
2     p_id_projet BIGINT,  
3     p_annee INTEGER  
4 ) RETURNS INTEGER
```

Listing 14 – Signature de la fonction

Implémentation

```
1 -- Code SQL complet
```

Listing 15 – Fonction nb_publications_projet

Tests

```
1 -- Scripts de test
```

Listing 16 – Tests de la fonction

6.2.2 Procédure 2 : Préparation du bilan d'un projet

Objectif Mettre à jour une table de bilan avec le nombre de publications et jeux de données par projet pour une année donnée.

Table de bilan

```
1 -- Structure de la table
```

Listing 17 – Table bilan_projet

Implémentation

```
1 -- Code SQL complet
```

Listing 18 – Procédure preparation_bilan

Tests Scripts de test et résultats.

6.2.3 Fonction 3 : Fiche projet

Objectif Renvoyer la liste des publications et jeux de données associés à un projet avec leurs principales informations.

Type de retour Description de la structure retournée (TABLE, SETOF, JSON, etc.).

Implémentation

```
1 -- Code SQL complet
```

Listing 19 – Fonction fiche_projet

Tests Exemples d'utilisation et résultats.

6.2.4 Procédure 4 : Archivage des contrats échus

Objectif Déplacer les contrats dont la date de fin est antérieure à une date seuil dans une table d'archives.

Table d'archives

```
1 -- Structure de la table
```

Listing 20 – Table contrat_archive

Implémentation

```
1 -- Code SQL complet
```

Listing 21 – Procédure archivage_contrats

Tests Scripts de test avec différentes dates seuil.

6.3. Documentation du code

6.3.1 Conventions de nommage

Description des conventions adoptées pour les variables, paramètres, etc.

6.3.2 Commentaires

Exemples de commentaires dans le code PL/pgSQL.

7. Conclusion

7.1. Synthèse du travail réalisé

Récapitulatif des différentes parties et des résultats obtenus.

7.2. Apports pédagogiques

7.2.1 Compétences techniques acquises

- Conception de schéma relationnel complexe
- Maîtrise de PostgreSQL et PL/pgSQL
- Techniques d'optimisation de requêtes
- Gestion de la sécurité et des accès

7.2.2 Utilisation de l'IA générative

Retour critique sur l'utilisation des outils d'IA dans le processus de développement.

7.3. Difficultés rencontrées

Liste des principaux défis et solutions apportées.

7.4. Perspectives d'amélioration

Pistes d'amélioration pour la base de données et le processus de développement.

A. Scripts SQL

A.1. Script de création du schéma

Référence au fichier `schema_univ_recherche.sql`.

A.2. Script d'insertion des données

Référence au fichier `inserts_postgres.sql`.

A.3. Script de gestion des utilisateurs

Référence au fichier `schema_avance.sql` Partie 3.

A.4. Scripts des requêtes optimisées

Référence au fichier `schema_avance.sql` Partie 4.

A.5. Scripts des triggers et procédures

Référence au fichier `schema_avance.sql` Partie 5.

B. Code Python

B.1. Script de génération de données

Référence au fichier `generate_data.py`.

C. Captures d'écran

C.1. Plans d'exécution

Captures d'écran des plans EXPLAIN ANALYZE pour les requêtes optimisées.

C.2. Tests des procédures

Captures d'écran des résultats des tests.

D. Dictionnaire de données

Référence au fichier `dictionnaire_attributs.md`.